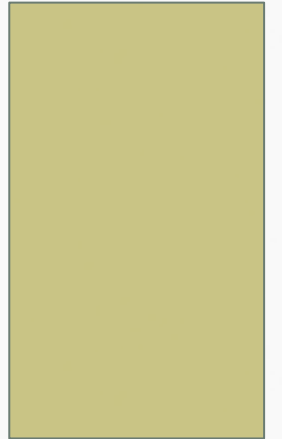


EC-200 DATA STRUCTURES

LECTURE 18



MERGESORT

- Divide and Conquer Algorithm
- Recursively split the input in half
- Then recursively merge pairs of pieces

Recursive Steps:

Divide the Array in Half

Sort the Halves

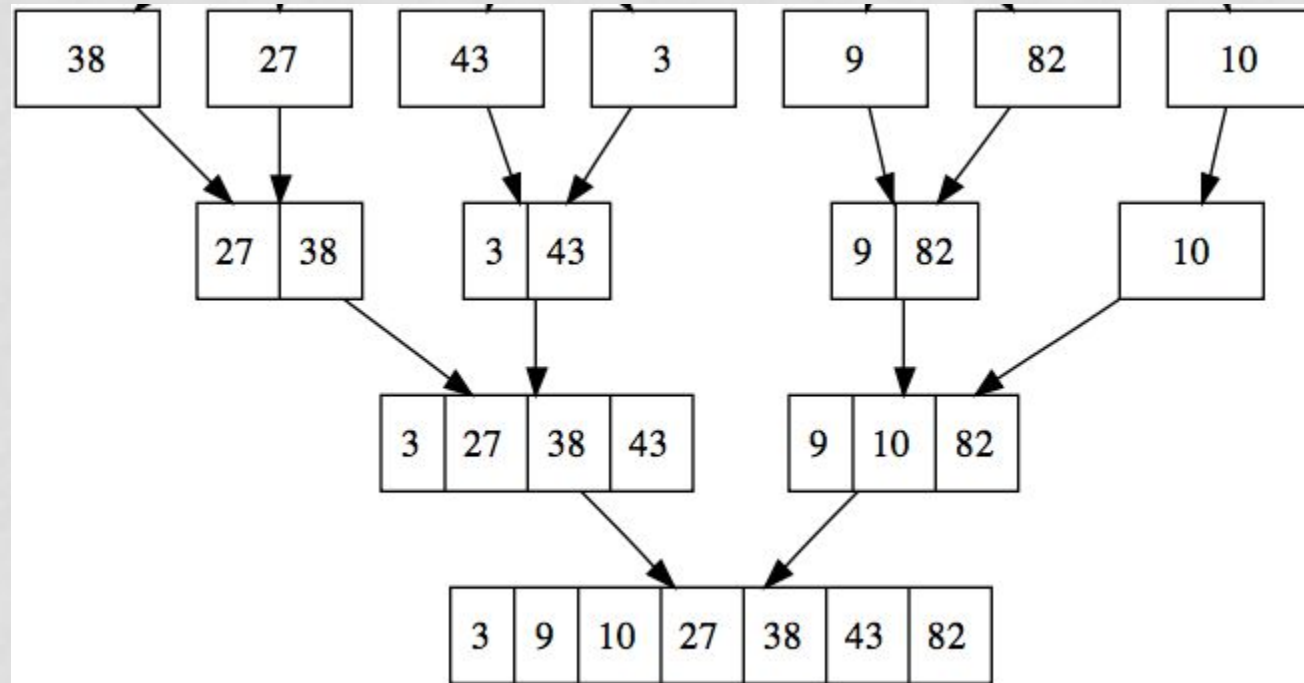
Merge The Halves inside a Temporary Array

Copy Temporary Array to the appropriate locations in original array

MERGESORT

- The Recursive calls Continue to divide the Array into Pieces Until Each Piece Contains Only One Item
- An Array of One Item is Obviously Sorted
- The Algorithm Then Merges These small Pieces Until One Sorted Array Results

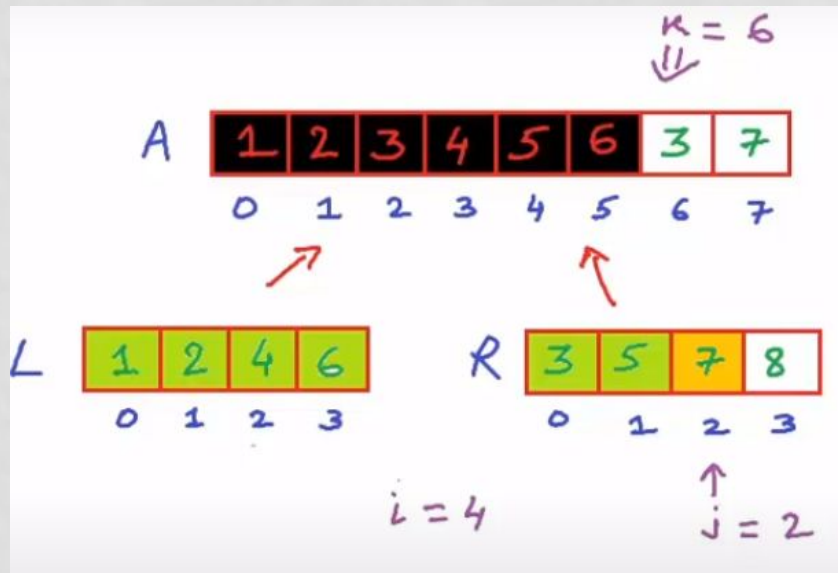
MERGE SORT



ALGORITHM

- Splitting arrays into L and R
- Merge them using the discussed algorithm

MERGE ARRAY



Merge (L, R, A)

```
{
  nL -> size of L; nR -> size of R;
  nA -> size of A;
  indL, indR, indA -> 0
  while ( indL < nL && indR < nR )
  {
    if( L[indL] <= R[indR] )
    {
      A[indA] -> L[indL];
      indL++; indA++;
    }
    else
    {
      A[indA] -> R[indR];
      indR++; indA++;
    }
  }
}
```

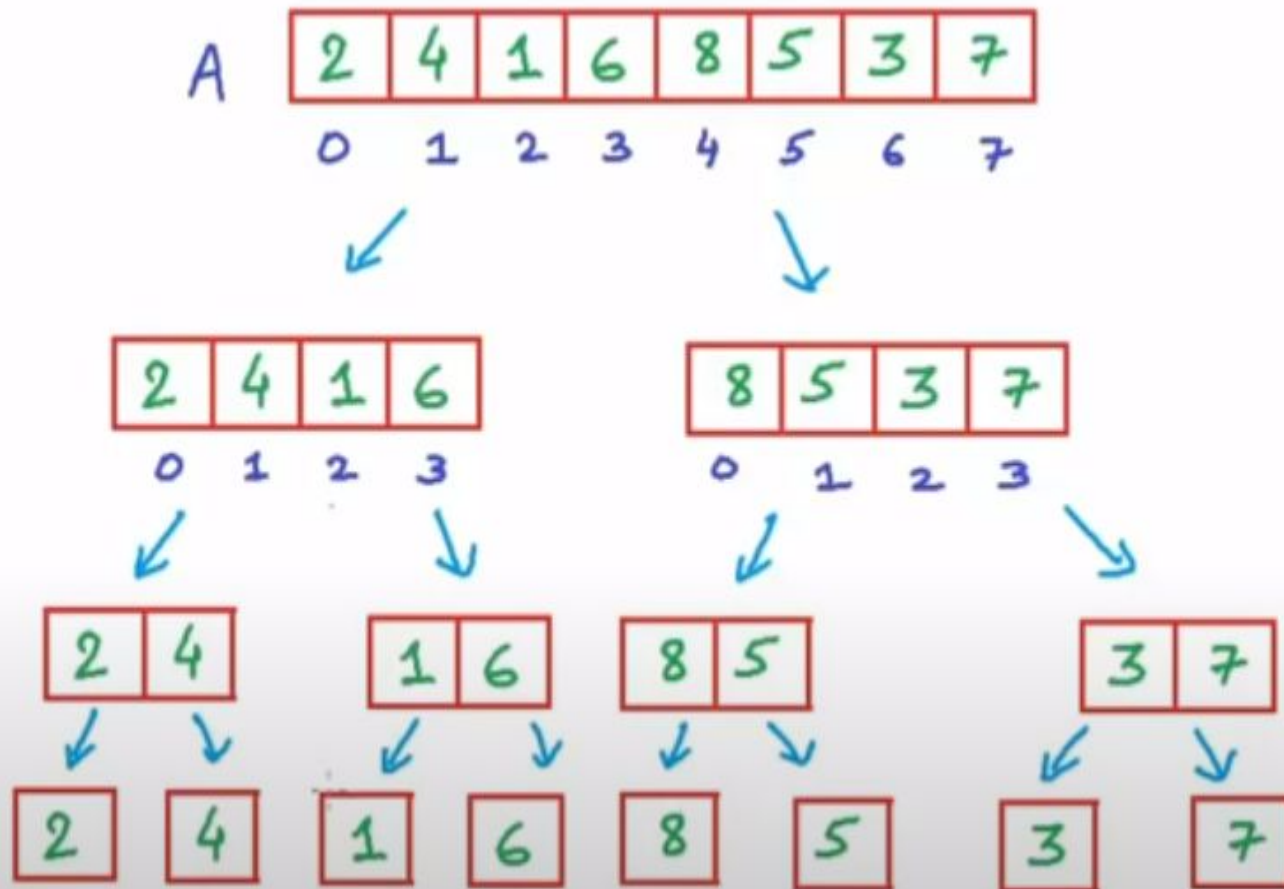
while(indL < nL)

```
{
  A[indA] -> L[indL];
  indL++; indA++;
}
```

while(indR < nR)

```
{
  A[indA] -> R[indR];
  indR++; indA++;
}
```

SPLIT ARRAY



```
MergeSort ( int n, int arr[ ] )
```

```
{
```

```
  if(n<2)
```

```
    return;
```

```
  mid = n/2;
```

```
  left -> new array of size [mid]
```

```
  right -> new array of size [n-mid]
```

```
  i = 0 -> mid-1
```

```
  left = arr[i];
```

```
  i = mid -> n-1
```

```
  right = arr[i];
```

```
  MergeSort(leftSize, left);
```

```
  MergeSort(RightSize, right);
```

```
  Merge(left, right, arr);
```

```
}
```

ANALYSIS

- There are a total of $\log_2 n$ passes
- Each pass requires atmost n comparison
- Complexity = $O(n \log_2 n)$

QUICKSORT

- **Divide and Conquer** algorithm
- Quicksort Works by Partitioning the Array into Two Pieces Separated by a Single Element That is Greater Than all the Elements in the Left part and Smaller Than all the Elements in the right part
- This Guarantees That, the Single Element , Called the **Pivot** Element, is in its Correct position
- Then the Algorithm Proceeds, Applying the Same Method to the Two parts Separately

QUICKSORT

- Partition (Divide)
 - Choose a **pivot**
 - Find the position for the pivot so that
 - all elements to the left are less
 - all elements to the right are greater

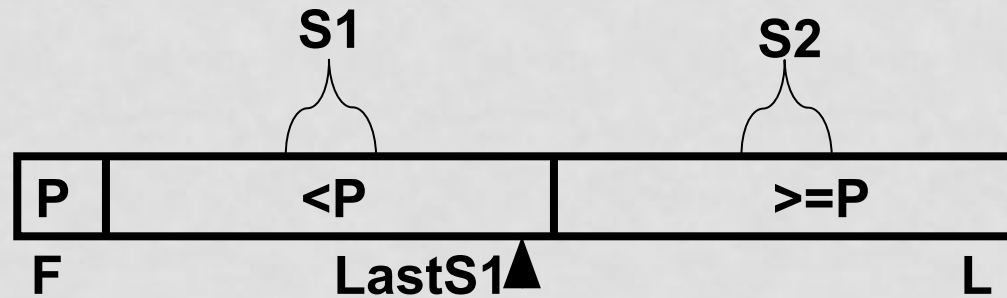


QUICKSORT

- Conquer
 - Apply the same algorithm to each half



HOW TO PERFORM THE MOVES



- **To Move Pivot Between S1 and S2**
 - **Interchange $A[\text{LastS1}]$ (i.e. the Rightmost Item in S1) with Pivot**
 - Thus, Pivot Would be in its Appropriate Location

Pivot					
27	38	12	39	27	16

Choose Pivot, here A[F],
LastS1=F

Pivot	Unknown				
27	38	12	39	27	16

FirstUnknown = 1(Index to 38
38 Belongs in S2)

Pivot	S2	Unknown			
27	38	12	39	27	16

S1 is Empty
12 Belongs in S1, swap38 & 12

Pivot	S1	S2	Unknown		
27	12	38	39	27	16

39 Belongs in S2

Pivot	S1	S2	Unknown		
27	12	38	39	27	16

27 Belongs in S2

Pivot	S1	S2	Unknown		
27	12	38	39	27	16

16 Belongs in S1, Swap 38 & 16

Pivot	S1	S2			
27	12	16	39	27	38

No more Unknown

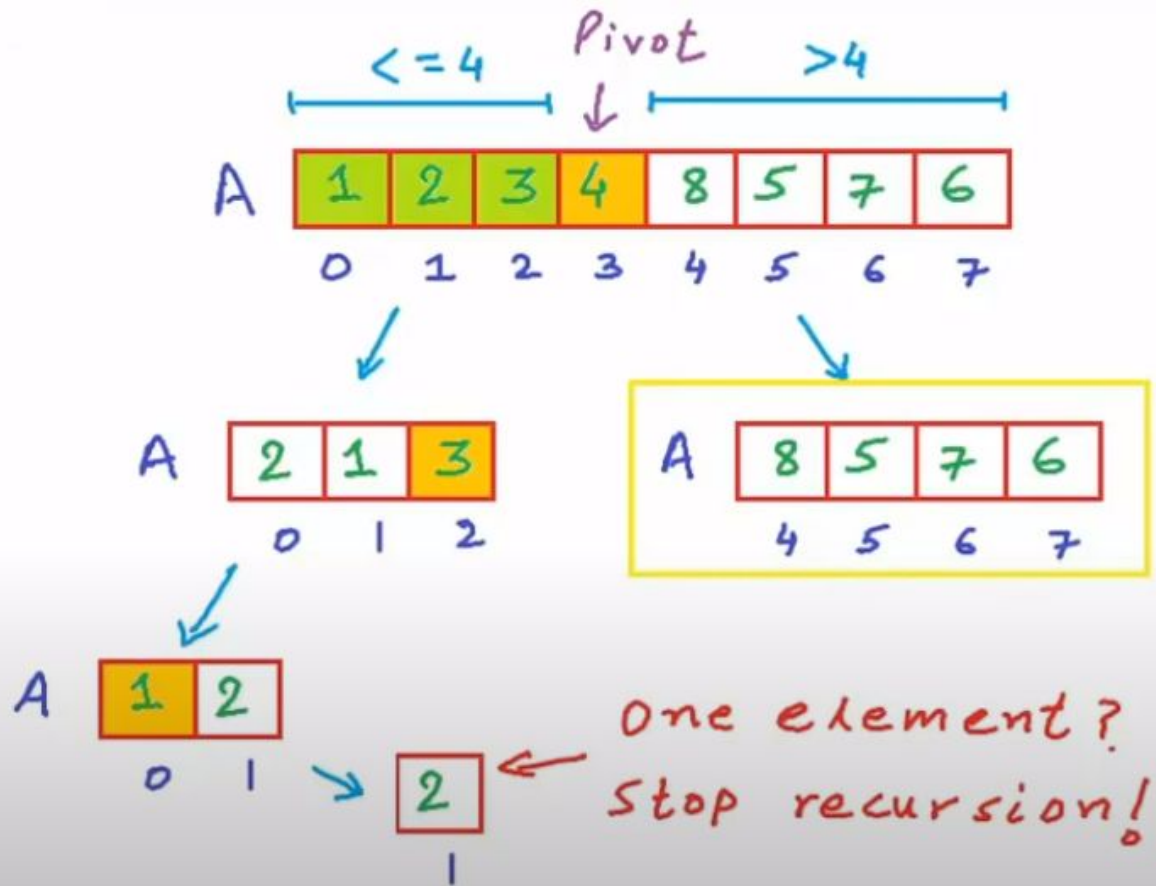
S1		Pivot	S2		
16	12	27	39	27	38

Place Pivot between S1 and S2

ALGORITHM

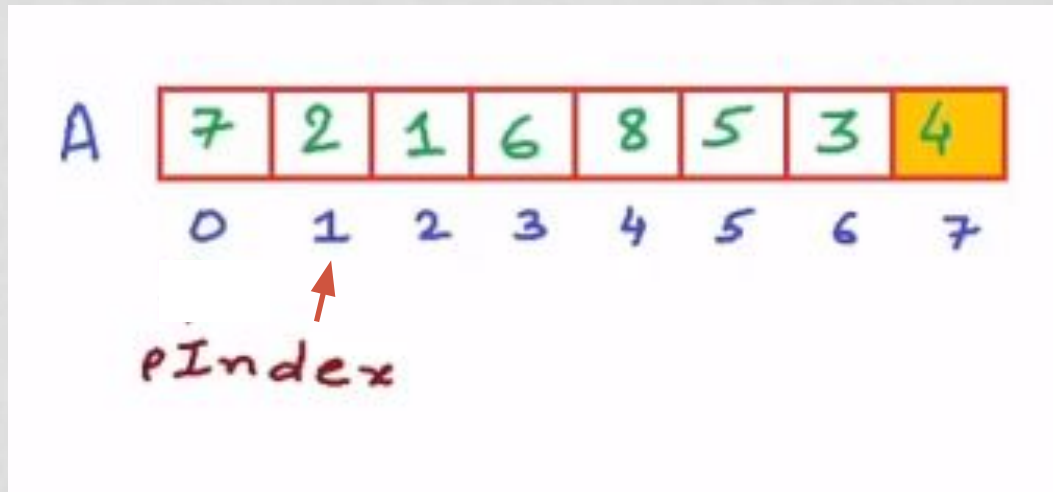
- Choose a pivot and re-arrange array w.r.t pivot
- Partition the array into two parts taking pivot as center

PARTITIONING



```
QuickSort (int arr[ ], int start , int end)
{
    if ( start >= end )
        return;
    plIndex=Partition (arr, start, end);
    QuickSort ( arr, start, plIndex-1 );
    QuickSort ( arr, plIndex+1, end );
}
```

RE-ARRANGING ARRAY



```
int Partition ( Arr [ ] , int start, int end )
{
    pivot = Arr [ start ];
    pIndex = start + 1;

    for i = start + 1 -> end
    {
        if ( Arr [ i ] < pivot )
        { swap ( Arr [ pIndex ], Arr [ i ] );
          pIndex++; }
    }

    swap( Arr [ pIndex ], Arr [ start ] );
    return pIndex;
}
```

REFERENCES

- <https://www.youtube.com/watch?v=TzeBrDU-JaY>
- <https://www.youtube.com/watch?v=COk73cpQbFQ&t=936s>